

SEECS
SCHOOL OF ELECTRICAL ENGINEERING & COMPUTER SCIENCE, NUST

Register File Verification with UVM RAL Model



EE-437: Digital Design Verification

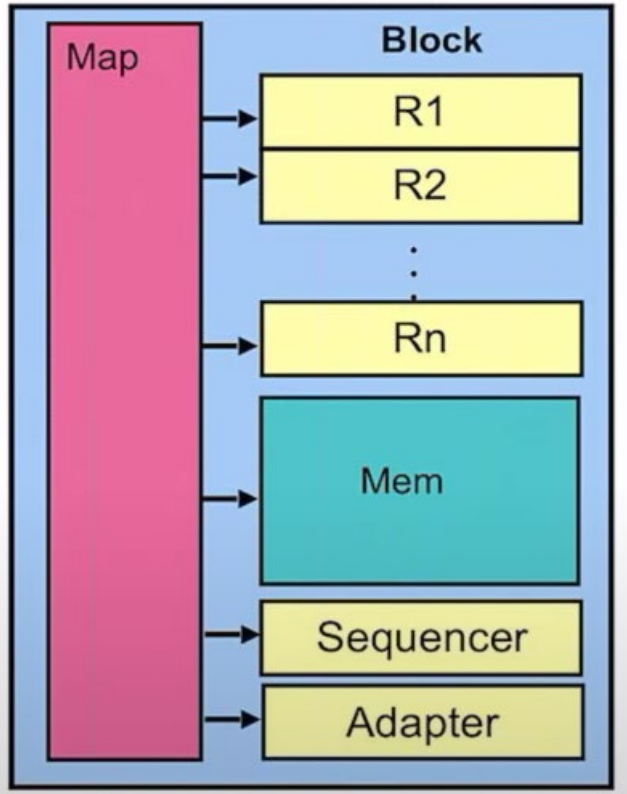
by Moonis Amir Zaman (365753)
Muhammad Bilal Khan (380046)

Overview

- **Goal:** To verify the functionality of a Register File using the UVM Register Abstraction Layer (RAL)
- **Key Objectives:**
  - Develop a UVM testbench with a robust register model
  - Implement comprehensive test sequences for register access
  - Verify both front-door (bus-based) and back-door (direct) register accesses
  - Ensure data integrity and correct register behavior
  - Collect functional coverage to measure verification completeness

Register Model

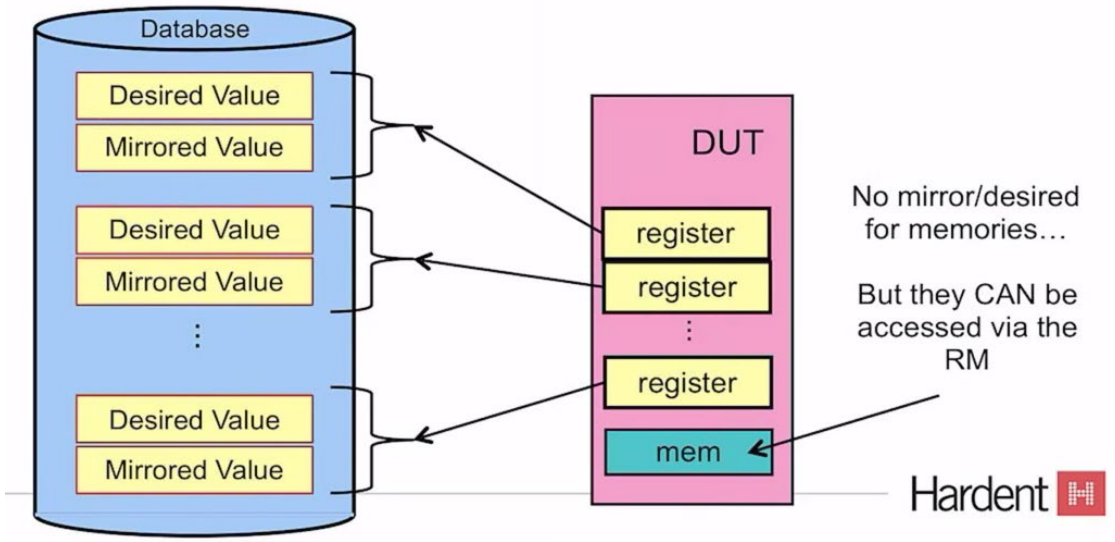
- UVM provides base classes to build a “register model”
  - Script generated, executable model, based on design HW/SW spec
  - Represents “just” the storage elements in your design
  - Convenient access to registers & memories of DUT via one or more bus interfaces
  - Provides a convenience layer for accessing DUT registers and memories



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

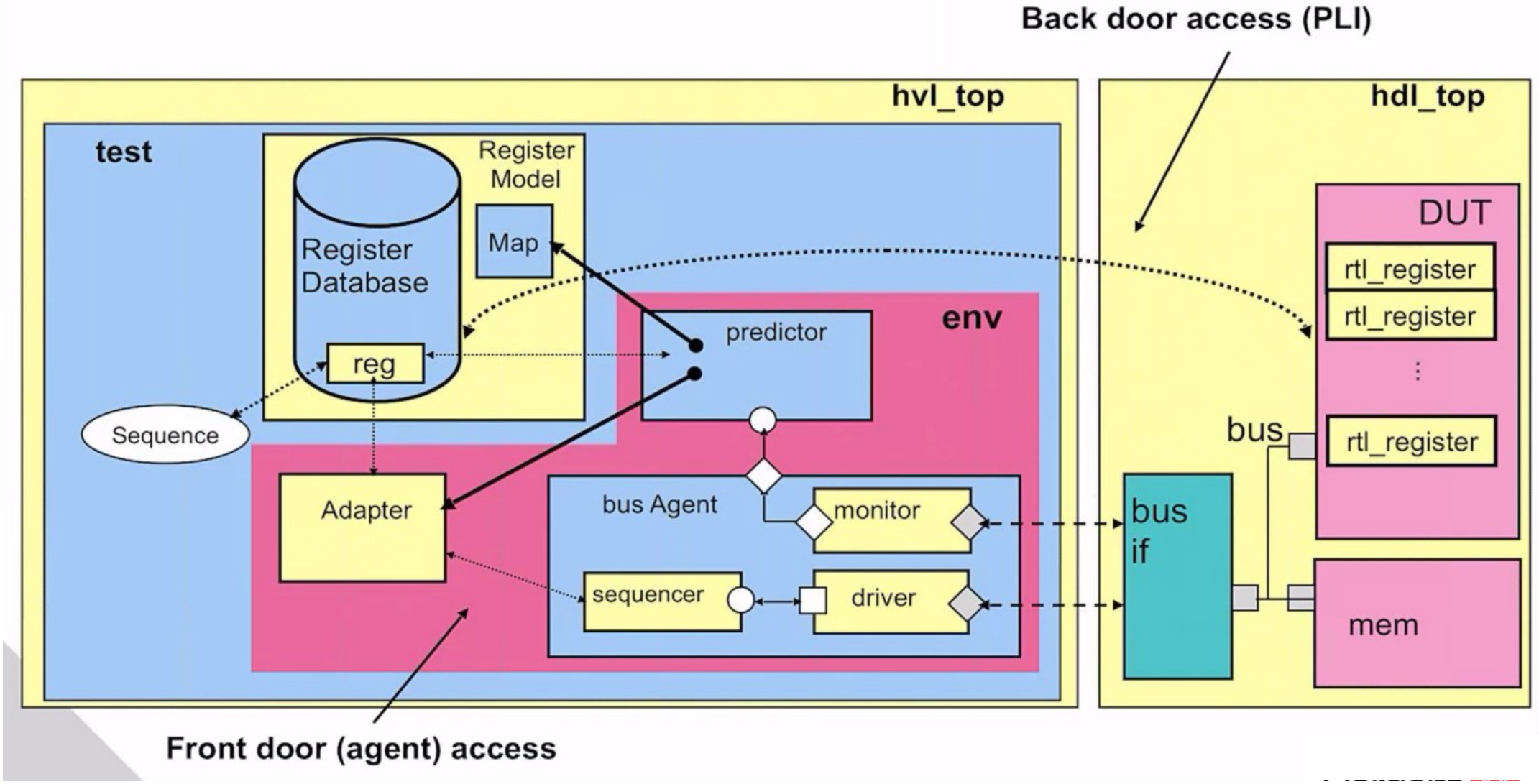
Register Database

- Register model has its own ‘database’
- Represents the state of the hardware registers
- Each DUT register is represented twice!
- **Desired Value** – what we WANT in the hardware reg next
- **Mirrored Value** – what we THINK is in the hardware reg now



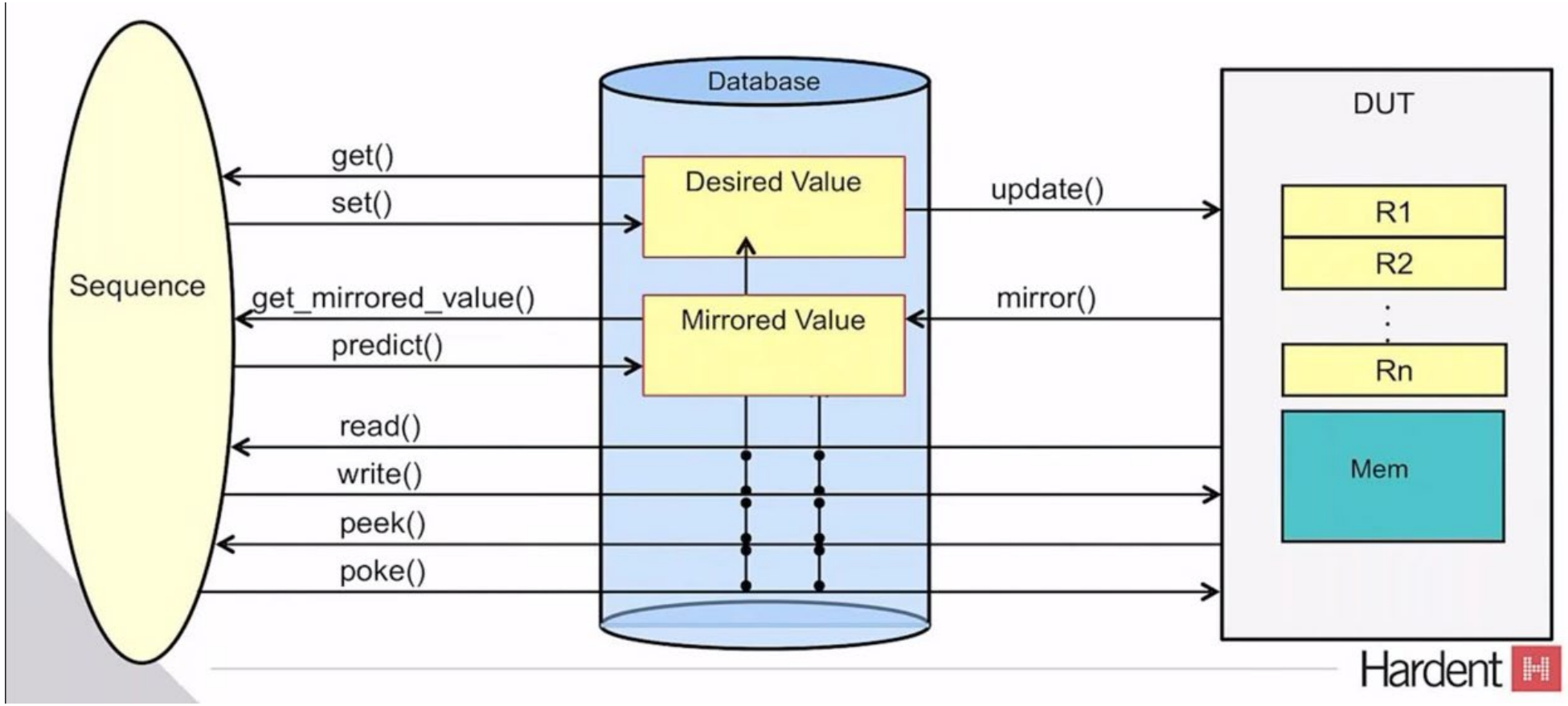
(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

Register Layer



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

Access Methods



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

Front-Door vs. Back-Door Access

Backdoor Access	Frontdoor Access
Take zero simulation time	Use a bus transaction which will take at least one RTL clock cycle
Write direction accesses force the HW register bits to the specified value	Write direction accesses do a normal HW write
Read direction accesses return the current value of the HW register bits	Read direction accesses to a normal HW read, data is returned using the HW data path
In the UVM register model, backdoor accesses are always auto-predicted - the mirrored value reflects the HW value	Frontdoor accesses are predicted based on what the bus monitor observes
Only the register bits accessed are affected, side effects may occur when time advances depending on HW implementation	Side effects are modelled correctly
By-passes normal HW	Simulates real timing and event sequences, catches errors due to unknown interactions

(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

Register Model Implementation



- **Individual Registers:**
  - Each DUT register (e.g., rxtx0, ctrl) modeled as a **uvm_reg** class.
  - Fields within registers defined as **uvm_reg_field** with attributes (size, LSB, access, reset).
- **Register Block:**
  - Container (**uvm_reg_block**) for all register objects.
  - Defines a **uvm_reg_map**:
  - Maps register objects to their addresses in the DUT.
  - Specifies bus width and endianness.
  - Configures HDL paths for back-door access.
- **Adapter:**
  - Translates generic RAL operations to bus-specific sequence items and vice-versa.
    - ❑ **reg2bus()**: RAL operation -> Bus transaction.
    - ❑ **bus2reg()**: Bus transaction -> RAL operation (for predictor).

Test Sequences

- **Primary Test Sequence:**
  - Comprehensive Register Testing: Iterates through all registers in the model.
  - For each register:
  - **Predefined Patterns:** (0x0000, 0xFFFF_FFFF, 0xAAAA_AAAA, etc.)
    - Front-Door Write -> Front-Door Read -> Back-Door Peek. Verify at each step.
  - **Random Value (Front-Door):**
    - Front-Door Write (random data) -> Front-Door Read -> Back-Door Peek. Verify.
  - **Random Value (Back-Door):**
    - Back-Door Poke (random data) -> Front-Door Read -> Back-Door Peek. Verify.
- **Field-Level Access Tests:**
  - Targeted specific fields (e.g., ss_reg.char_len).
  - Verified using uvm_reg_field::write() and uvm_reg_field::poke().
  - Confirmed field value and impact on the full register.

Test Sequences

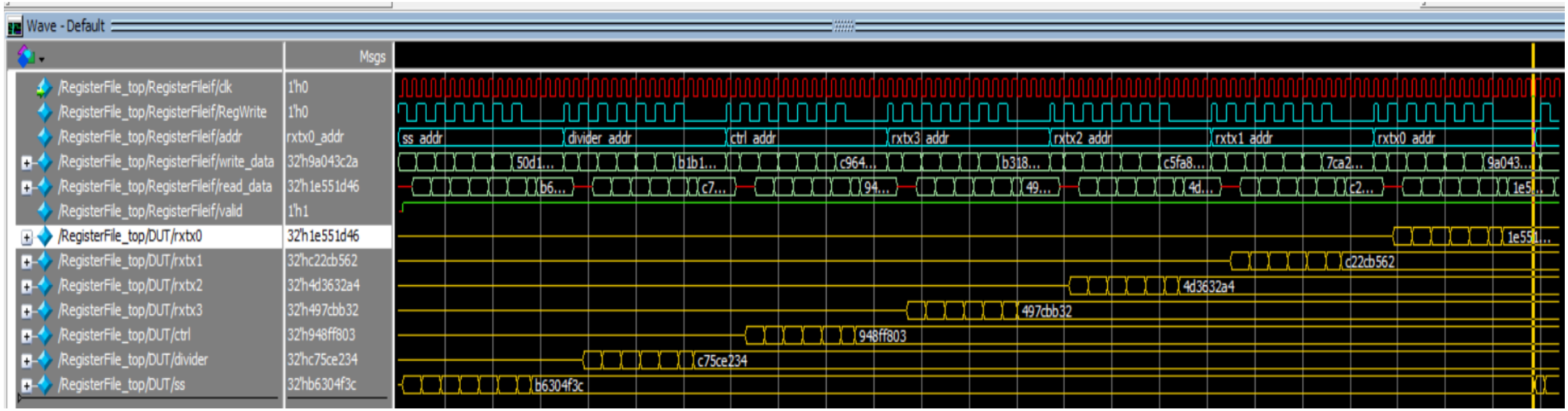
Test Case ID	Test Case Name	Description	UVM Sequence(s) Used	Key Features Verified	Stimulus Details	Expected Outcome	Priority	Status
REG_ACCESS								
Pattern test	Comprehensive RAL Access	Verifies R/W via FD & BD for all registers and fields using various data patterns. Checks model synchronization.	`RegisterFile_reg_seq`	VREG_RW, VREG_BD, VREG_FD, VREG_DATA, VREG_RAL_SYNC	All registers; patterns (0, all 1s, walking, random); FD/BD R/W;	All RAL operations (write/read/poke/peek/field ops) complete with UVM_IS_OK. Read/peeked data matches written/poked data. Mirrored values are correct.	High	Implemented
STRESS_TESTING								
Randomized test	RAL Random Stress Test	Subjects registers to a high volume of randomized read/write operations via FD.	Custom random sequence in `RegisterFile_reg_seq`	VREG_RW, VREG_DATA, VREG_RAL_SYNC	N (large number) random FD R/W operations to random valid addresses with random data.	All operations complete with UVM_IS_OK. Data integrity maintained. Model synchronized. No unexpected DUT behavior.	High	Implemented
FIELD_TESTING								
ss_reg field	RAL Field Test	Write/Poke JUST the char_len field of ss_reg	Test case 4 (label) in `RegisterFile_reg_seq`	VREG_FIELD	Change character field of ss_reg	Only the char_len field should be modified	Low	Implemented

Functional Coverage

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/RegisterFile_monitor_pkg/RegisterFile_monitor		100.00%				
TYPE RegisterFile_CG		100.00%	100	100.00%		
CVP RegisterFile_CG::cp_addr		100.00%	100	100.00%		
CVP RegisterFile_CG::cp_reg_write		100.00%	100	100.00%		
CVP RegisterFile_CG::cp_ss_char_len		100.00%	100	100.00%		
CVP RegisterFile_CG::cp_valid_state		100.00%	100	100.00%		
CROSS RegisterFile_CG::cross_addr_rw		100.00%	100	100.00%		
INST MonitorCoverageInstance		100.00%	100	100.00%		

Waveform Simulation



Conclusion

- UVM RAL provides a powerful and efficient framework for register verification.
- The combination of register model, adapter, predictor, and well-defined sequences enables thorough testing.
- Front-door and back-door accesses test different aspects and are both vital.
- Functional coverage is essential to guide verification efforts and ensure completeness.
- This project successfully applied UVM RAL to verify the RegisterFile DUT.



Thank You